STAT405 Final Report

Lily Wang (62800495), Sky Yun (55333710), Zhengxi Yuan (15025604)

2026-03-23

Introduction

Our team worked with two U.S. housing-related datasets to study how well bayesian methods can predict housing prices in the US. The main goal of this project is to compare a Bayesian linear regression model with a time-series model, to determine which model better predicts quarterly median U.S. house prices.

The response dataset is the "Quarterly Median U.S. House Prices" dataset, and the explanatory dataset is the "Economic Predictors for House Prices" dataset, which consists monthly economic and housing-related factors. Based on our proposal, we decided to include the following variables:

- `MSPUS` (Response variable): the quarterly median sales price of houses sold in the United States.
- `GDP`: how the US economy impacts housing prices.
- `mortgage_rate`: Percentage interest charged on a home loan.
- `unemployment_rate`: Percentage of those unemployed in the US economy.

Since the response variable is recorded quarterly while the economic predictors are recorded monthly, the monthly predictor data must be aggregated to the quarterly level before merging with the quarterly housing price data. After merging the two data sets together, we checked for missing values and found no missing observations. Since we are doing a Bayesian analysis, it is hard to explore the posterior regions efficiently enough due to large scales. This analysis uses scaled values to make it easier to put priors on our model parameters.

Table 1: US Median Housing Prices

MSPUS	DATE	GDP	mortgage_rate	unemployment_rate	time_index
186000	2003-01-01	14614.14	5.834500	5.866667	1
191800	2003-04-01	14743.57	5.508833	6.133333	2
191900	2003-07-01	14988.78	6.014667	6.133333	3
198800	2003-10-01	15162.76	5.920167	5.833333	4
212700	2004-01-01	15248.68	5.597500	5.700000	5
217600	2004-04-01	15366.85	6.129167	5.600000	6

Literature Review

Previous study on the Median US Housing Price written by Yuxiao Chen argued that during the federal reserve's continuous interest rate increases in 2022-2023, the US house prices were expected to drop, but not fall sharply. The results showed that US house prices were still increasing, but at a lower rate. First, mortgage rates don't depend solely on interest rates, it is influenced more by inflation expectations and US

federal demand for US bonds. Also, housing prices are sticky, so they may take years to adjust to market changes.

This previous study focuses on economic argument with literature discussion, while our analysis is much more statistical and Bayesian. We build Bayesian models to predict and explain median US housing prices using macroeconomic factors, like GDP, unemployment rate, and mortgage rate. The study done by Yuxiao Chen relies more on economic reasoning and historical discussions.

Simple Linear Regression Model

Our naive model:

$$y_t = \beta_0 + \beta_1 \,\text{time}_t + \beta_2 \,\text{GDP}_t + \beta_3 \,\text{UnemploymentRate}_t + \beta_4 \,\text{MortgageRate}_t + \varepsilon_t$$

Our prior and likelihood:

$$\begin{aligned} \beta_t &\sim \mathcal{N}(0, 10), \quad t \in \{0, 1, 2, 3, 4\} \\ \sigma &\sim \text{Exponential}(1) \end{aligned}$$

$$\begin{aligned} y_t|\mu_t &\sim N(\mu_t, \sigma) \\ \mu_t &= \beta_0 + \beta_1 \,\text{time}_t + \beta_2 \,\text{GDP}_t + \beta_3 \,\text{UnemploymentRate}_t + \beta_4 \,\text{MortgageRate}_t \end{aligned}$$

Simple Linear Regression Model with SNIS

After initializing and running the SNIS algorithm with re-sampling, we came to the following outputs. The calculated ESS and posterior mean for each parameter are:

```
## [1] 3067.872
```

```
##       (Intercept)        time_index               GDP unemployment_rate
##      0.0002769322     -0.0419072113      1.0865831532     -0.0295095054
##     mortgage_rate             sigma
##      0.1681124238      0.2714782414
```

Summary of the posterior results:

Table 2: BLR SNIS Posterior Summary Table

	parameter	mean	sd
(Intercept)	(Intercept)	0.0001852	0.0296519
time_index	time_index	-0.0466367	0.2972513
GDP	GDP	1.0930261	0.2978301
unemployment_rate	unemployment_rate	-0.0283001	0.0621688
mortgage_rate	mortgage_rate	0.1701407	0.0544291
sigma	sigma	0.2716139	0.0224433

Doing posterior predictive checks, we get the following:

Table 3: BLR SNIS Posterior Predictive Checks

statistic	observed	replicated_mean	bayesian_p_value
mean	0	0.0005826	0.5032
sd	1	1.0046517	0.5362

We fitted a Bayesian linear regression model to standardized house prices (MSPUS) using time_index, GDP, unemployment_rate, and mortgage_rate as predictors. Posterior inference was conducted using self-normalized importance sampling (SNIS).

The effective sample size of our SNIS approximation is 3067.872 out of 10,000 draws. This suggests that the proposal distribution matched the posterior, stating that the posterior approximation is stable.

Looking at the posterior mean coefficients, GDP seems to be the strongest predictor, with a value of 1.09. The other predictors show relatively small positive and negative effects on median US housing price. The posterior mean of the residual standard deviation is around 0.27, suggesting there are still some unexplained variation remaining.

Posterior predictive checks indicate that the model fits the data reasonably well. The replicated mean and observed mean are both close to 0 and the standard deviations are similar. Also, the Bayesian p-values are near 0.5. In addition, the predictive plots show that the model captures the overall upward trend in house prices.

Overall, this model provides a good Bayesian baseline for the house-price data. It yields stable posterior inference, identifies GDP as the main predictor, and captures the main pattern in the data.

Simple Linear Regression Model with MCMC

MCMC Sampling Results

We ran the MCMC sampler for 10000 iterations with 2500 warm-up iterations to ensure that the MCMC chain is sufficiently burned-in. Based on the posterior mean, GDP has the largest coefficient, which is consistent with the the results found in posterior approximation using SNIS. It appears that the associated standard deviations for the paramters are slightly larger than the ones given my SNIS which may indicate that the there is slightly more uncertainty involved when estimating the parameters using the MCMC method.

Table 4: Posterior Summary for BLR MCMC

variable	mean	median	sd	q5	q95
beta0	-0.0001467	-0.0000598	0.0303118	-0.0502209	0.0497787
beta1	-0.0574083	-0.0559020	0.3084883	-0.5617861	0.4495917
beta2	1.1047440	1.1051646	0.3078587	0.5962371	1.6051579
beta3	-0.0250531	-0.0249005	0.0635883	-0.1290985	0.0789599
beta4	0.1701332	0.1703378	0.0551467	0.0782762	0.2607785
sigma	0.2717578	0.2701526	0.0226090	0.2376397	0.3111587

MCMC Diagnostics: Posterior Predictive Checks and Trace Plots

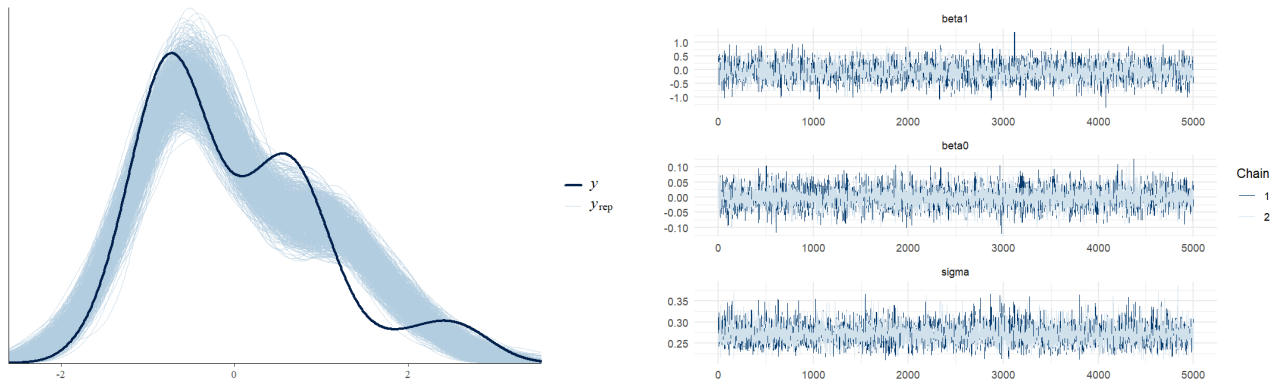


Figure 1: Posterior Predictive Checks and MCMC Mixing Diagnostics

We can observe above that the posterior predictive samples sufficiently capture most of the variation of the observed data but struggles to fit the smaller peaks. The trace plots for σ and β_0, β_1 between the two chains are somewhat indistinguishable from each other and there are no observable trends or patterns, suggesting fast-mixing in our MCMC algorithm. Repeated trials provided similar results.

One-Step Forecast Assessment

We will compute mean absolute scaled error (MASE) using our posterior predictive samples to assess the accuracy of our one-step forecast predictions. As usual, we exclude the last observation in our time series to use as our test point and then train the model on the rest of the data.

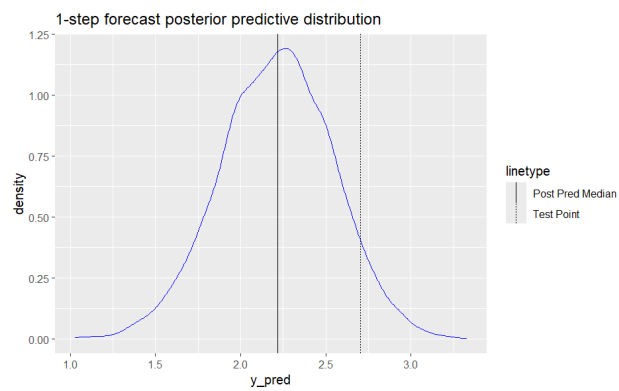


Figure 2: Posterior Predictive Sample vs Test Point

Table 5: Prediction Summary

obs_value	mase	median	q5	q95
2.705354	1.541413	2.218976	1.65904	2.74827

We can see from the plot above that the observed test point is somewhat far from the median, with a MASE value of 1.541. However, this is not unexpected as we have already seen before the model struggles to capture

abrupt changes in trends near the end of the series. This may suggest that simple BLR might not be the most suitable model for obtaining accurate forecast, leading us to the next model.

BLR SNIS VS. MCMC

We compared SNIS and MCMC for the same baseline Bayesian linear regression model. The two methods produced very similar posterior results. The posterior means for SNIS and MCMC were 0.0003 and -0.0003 for the intercept, -0.0419 and -0.0660 for time_index, 1.0866 and 1.11 for GDP, -0.0295 and -0.0233 for unemployment_rate, 0.1681 and 0.170 for mortgage_rate, 0.2715 and 0.271 for sigma. In both methods, GDP was the strongest positive predictor.

The posterior uncertainty estimates were also close. For example, the posterior standard deviation of GDP was about 0.298 under SNIS and 0.313 under MCMC, while the posterior standard deviation of sigma was about 0.0224 under SNIS and 0.0226 under MCMC. This suggests that SNIS approximated both the center and spread of the posterior well.

The predictive results were very similar. For the first observation, SNIS gave a predictive mean of about -1.635, while MCMC gave a predictive mean of about -1.640. Both methods also produced fitted values that followed the overall upward trend in house prices. Overall, both methods performed well and pretty similar.

Time-Series MA(3)

Our second model is a time-series model that only depends on past-lagged values, capturing the dynamic structure of the data, including trends, seasonality, and time dependence. To find the best-fit model based on AIC, we ran `auto.arima()` and it suggested the moving Average model with three lagged forecast error terms (MA(3)) to be the best. We performed first-order differencing of the scaled median housing price to align with the stationary assumptions of moving-average models.

Assume that N is the size of the observed data y. We define the following Bayesian MA(3) model, where the right side describes our transformed parameters and likelihood distribution and the left defines our priors.

$$
\begin{aligned}
e_1 &= y_1 - \mu & \mu &\sim \text{Normal}(0,1) \\
e_2 &= y_2 - (\mu + \theta_1 e_1) & \theta_k &\sim \text{Normal}(0,0.5) \quad \text{for } t = 1,2,3 \\
e_3 &= y_3 - (\mu + \theta_1 e_2 + \theta_2 e_1) & \sigma &\sim \text{Cauchy}(0,1) \\
e_t &= y_t - (\mu + \theta_1 e_{t-1} + \theta_2 e_{t-2} + \theta_3 e_{t-3}) \\
Y_t|\sigma, \mu, e_{t-1}, e_{t-2}, e_{t-3} &\sim \text{Normal}\left(\mu + \sum_{\mathrm{i}=1}^{3} \theta_{\mathrm{i}} \mathrm{e}_{\mathrm{t-i}}, \sigma\right) \quad \text{for } t = 4, ..., N
\end{aligned}
$$

Priors (and associated hyper-parameters) were chosen to reflect the general range and scale of our parameters of interest. For example, σ is only allowed to take positive, real numbers so exponential and half-normal distribution were used as priors initially but proved to be not informative enough for the MCMC the converge reliably. Additionally, to ensure stationarity (refer to appendix for more information), θ values should be within the range $[-1, 1]$. So a normal distribution was chosen with a small standard deviation to reflect the symmetric bounds around 0.

MA(3) with MCMC

MCMC Sampling Results

It was quite difficult to sample from this MCMC chain due to the non-identifiability of the MA model. Putting more restrictive priors on the parameters helped minimize divergences. We iterated 10000 MCMC samples with 2500 warm-up iterations to sufficiently burn-in the MCMC chain. The clean summary is provided below.

Table 6: Posterior Summary for MCMC MA(3) Model

variable	mean	median	sd	q5	q95
mu	0.0525884	0.0524052	0.0215693	0.0173870	0.0880828
theta[1]	-0.1572260	-0.1573797	0.1090876	-0.3342211	0.0225481
theta[2]	0.1626236	0.1642053	0.0994483	-0.0033502	0.3257414
theta[3]	0.3019928	0.3048143	0.1066392	0.1237636	0.4734586
sigma	0.1424103	0.1415569	0.0121492	0.1240869	0.1634912

It appears that posterior means of the parameters is somewhat consistent to the results given found in the itme series analysis. The largest coefficient θ_3 seems to indicate that the e_{t-3} is generally a strong predictor for the Y_t and all θ's show similar credible interval widths. However, there is more uncertainty involved when estimating the θ coefficients compared to μ and σ, as suggested by the narrow credible intervals.

MCMCM Diagnostics: Posterior Predictive Checks and Trace Plots

In figure, posterior predictive checks show that the nominal coverage (81.33%) of leave-one-out 80% credible intervals is sufficiently close to the actual coverage, suggesting that the model is fitting to the data well. Additionally, the trace-plots for for σ, μ and θ_1 fluctuates rapidly around the posterior means which indicates that there is fast-mixing between chains. Trace plots for θ_1 and θ_2 can be observed in the appendix. Same results were shown through repeated trials.

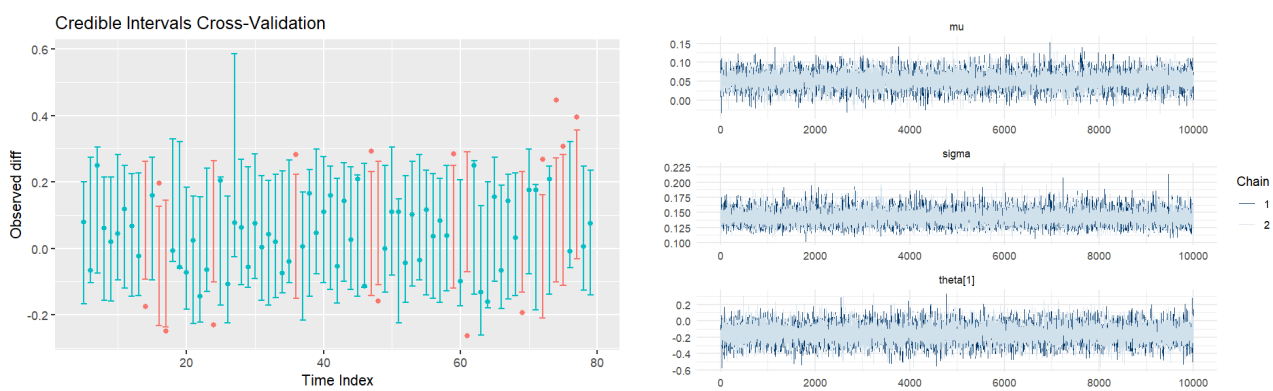


Figure 3: Posterior Predictive Checks and MCMC Mixing Diagnostics

One-Step Forecast Assessment

Just like before, we leave the last value out and train the model on the rest of the data and randomly sample from the posterior predictive distribution for the 1-step forecast, resulting in the following sample distribution and prediction summaries.

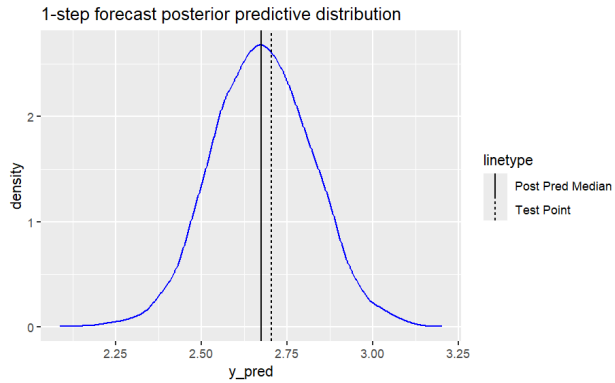


Figure 4: Posterior Predictive Sample vs Test Point

Table 7: Prediction Summary

obs_value	mase	median	q5	q95
2.705354	0.8103602	2.67614	2.437634	2.915123

The sample distribution is approximately normal as expected and we can observe that the test point is quite close to the posterior predictive median, suggesting that the samples are approximating the one-step forecast well. From figure 4. We can observe from the summaries that the MASE of 0.8104 is much lower compared to the Bayesian linear regression case with a MASE of 1.541.

MA(3) with Variational Inference

The following table produces the VI posterior summaries for the model parameters. Each parameter is presented with its mean, median, standard deviation, median absolute deviation, and the 5th and 95th posterior quantiles.

Table 8: MA(3) VI Posterior Summary Table

variable	mean	median	sd	mad	q5	q95
mu	0.0568412	0.0570456	0.0221101	0.0218440	0.0203187	0.0926144
sigma	0.1501529	0.1497649	0.0126425	0.0125934	0.1305411	0.1719556
theta[1]	-0.1497037	-0.1517660	0.1065219	0.1084351	-0.3199534	0.0310490
theta[2]	0.1754835	0.1776931	0.0944259	0.0949418	0.0159492	0.3299673
theta[3]	0.3223164	0.3254494	0.1047176	0.1060515	0.1437905	0.4873258

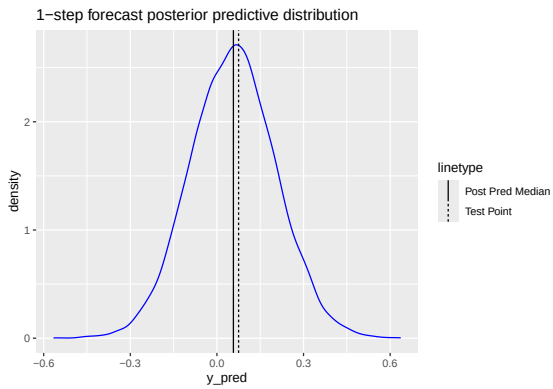
The intercept μ (mean = 0.0569, 90% CI: 0.020–0.093) captures a small upward drift in quarterly US house price changes. The entire credible interval above zero states that this trend is not just white noise.

θ_3 from the model has the largest positive mean value (mean = 0.322, 90% CI: 0.144–0.487), which indicates that shocks from three quarters ago pushes quarterly house price in the same direction. θ_2 shows a weaker effect at 0.175, while θ_1 suggests negative effect with a value of -0.150. The noise term σ (mean = 0.150, 90% CI: 0.131–0.172) is not too large, indicating the model is fairly stable.

One-Step Forecast

Similar to before, we randomly sample from the posterior distribution for the one-step forecast using VI method. Below are sample distribution and prediction summaries.

```
## [1] 0.8010144
```



The sample distribution is roughly normal, with the test point being very close to the posterior predictive mean. This suggests that the samples well approximate the one-step forecasts. Our VI MASE value of 0.80 is very close to our MCMC method, indicating that the two methods provide nearly identical accuracy.

Time-Series MCMC VS. VI

Refer to Table 6 and Table 8. Comparing the two posterior summary tables for the MA(3) model, the parameters estimates for the two methods are almost identical. Both methods indicate a negative first lag effect, and positive second and third lag effects, with θ_3 being the largest.

For forecasting performance, the two methods are essentially equivalent, with MCMC MASE value of 0.8104 and VI MASE value of 0.8010. This suggests that VI is a great approximation method, as it has minimal loss in predictive accuracy, and is much faster than the MCMC method.

Conclusion

BLR uses several explanatory variables to explain US median house prices, while MA(3) uses the series' past error to determine temporal dependence. BLR is a regression model, and MA(3) is a time-series model.

These two models have very different structures. For BLR, we can determine the covariate that has the largest effect, which is GDP under both SNIS and MCMC. For MA(3), we find the lag that has the largest effect, which is θ_3.

Looking at each model's forecasting performance, BLR's one-step-ahead MASE is about 1.54, and 0.81 with MCMC, 0.80 with VI for MA(3) model. Since lower MASE is better, we conclude that MA(3) performs better than BLR for forecasting.

Limitations

The MA(3) model uses only past forecast errors to predict future prices, ignoring economic factors. Also, this model is not good at predicting far future values as it doesn't have any information long-run trends. Lastly, the dataset is relatively small. With a sample size of only 80, our one-step forecasts may not be accurate compared to the true holdout values.

Contributions

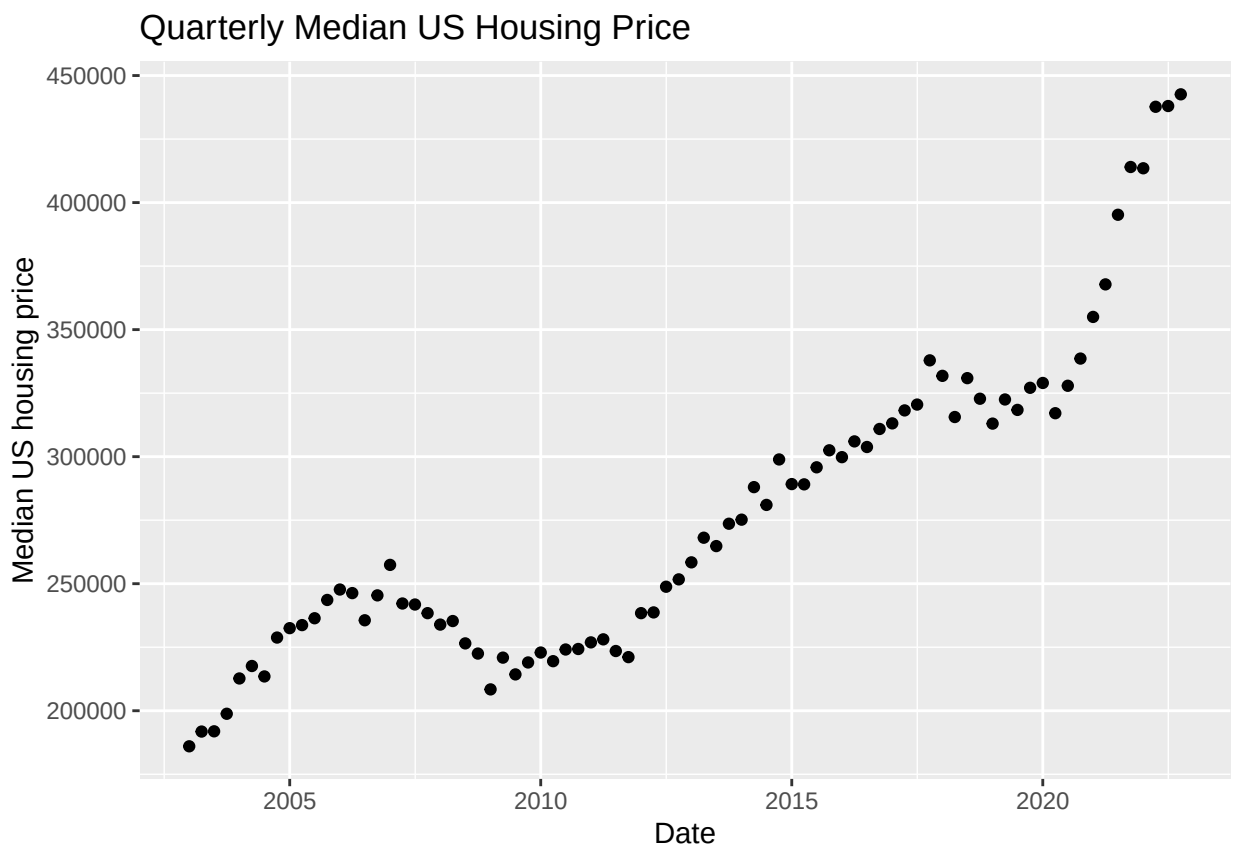
- Lily: Worked on SNIS and VI implementation and forecasting for the two models. Wrote the literature review, did a comparison between the VI and MCMC methods for the MA(3) model. Contributed to writing and formatting the final report.
- Jason: Worked on SNIS and VI implementation and forecasting for the two models. Did a comparison between the SNIS and MCMC methods for the BLR model, and an overall comparison between the BLR and MA(3) models.
- Sky: Composed MA3 model prior and likelihood specifications and worked on both MCMC methods for BLR and MA(3) Model. Developed STAN scripts and general scripts for MCMC diagnostics and forecasting predictions. Contributed to individual interpretations of the diagnostics, sampling and forecasting values for MCMC methods.

Appendix

Exploratory Analysis

Looking at the scatter plot, there seem to be a strong upward trend in date and GDP, Indicating that there is a positive correlation between GDP and date. From the correlation matrix, we see that GDP and Median US housing price has strong positive correlation value of 0.95. Both mortgage rate and unemployment rate are negatively correlated with Median US housing price.

```r
source("../source/data_sources_and_dependencies.R")
ggplot(dataset, aes(x = DATE, y = MSPUS)) +
  geom_point() +
  labs(x = "Date", y = "Median US housing price") +
  ggtitle("Quarterly Median US Housing Price")
```




```
numeric_data <- dataset[, sapply(dataset, is.numeric)]
cor_matrix <- cor(numeric_data)
corrplot(cor_matrix,
         method = "color",
         type = "upper",
         addCoef.col = "black",
         tl.col = "black",
         tl.srt = 45)
```

Stationarity and Reasoning Behind MA(3) Models

The MA(3) Model only applies to stationary series. Since our original scaled median house price time series shows a long-term increasing trend, differencing the series by the 1-lagged value can help make the series approximately stationary in order to fit an appropriate $MA(q)$ model. When we want to forecast for our original series, we then undifference the predictions given the MA(3) model by adding the previous lagged-value to the predictions.

Model Code

All associated code for this project are included in the working directory of the github repository. Each file either corresponds to specific model and approximation method or a type of exploratory analysis.

Model Code

- `MCMC_blr.Rmd`: MCMC Method for Bayesian Linear Regression
- `MCMC_ma3.Rmd`: MCMC Method for MA(3) Model
- `SNIS_blr.Rmd`: SNIS Method for Bayesian Linear Regression Model
- `VI_ma3.Rmd`: Variational Method for MA(3) Model

Exploratory Analysis

- `exploratory_analysis`: Predictor Analysis on GDP, Unemployment Rate, Time, Mortage Rate
- `time_series_analysis`: Time Series Analysis: Stationarity and best ARIMA model based on AIC

STAN Code and Associated for MCMC and VI

BLR Model MCMC

- BLR STAN Code for Sampling

```
data {
  int N;
  vector[N] y;
  vector[N] time;
  vector[N] gdp; // GDP
  vector[N] unempl_rate; //Enempoyment Rate
  vector[N] mortgage_rate;//mortgage rate
}

parameters {
  real beta0;
  real beta1;
  real beta2;
  real beta3;
  real beta4;
  real<lower=0> sigma;
}

model {
  beta0 ~ normal(0, 10);
  beta1 ~ normal(0, 10);
  beta2 ~ normal(0, 10);
  beta3 ~ normal(0, 10);
  sigma ~ exponential(1);

  for (i in 1:N) {
    real mu_i = beta0 + beta1*time[i] + beta2*gdp[i] + beta3*unempl_rate[i] + beta4*mortgage_rate[i];
    y[i] ~ normal(mu_i, sigma);
  }
}

generated quantities {
  vector[N] y_pred;

  for (i in 1:N) {
    y_pred[i] = normal_rng(
      beta0 + beta1*time[i] + beta2*gdp[i] + beta3*unempl_rate[i] + beta4*mortgage_rate[i],
      sigma);
  }
}
```

- Associated R Code for MCMC Sampling of BLR Model in `MCMC_blr.Rmd`

```r
source("source/data_sources_and_dependencies.R")

# Global Variables
n_samples = 10000
```

```r
y_scaled = data_scaled$MSPUS
seed=2026

train_data=list(
  N = length(y_scaled),
  y = y_scaled,
  time = data_scaled$time_index,
  gdp = data_scaled$GDP,
  unempl_rate = data_scaled$unemployment_rate,
  mortgage_rate = data_scaled$mortgage_rate
)

blr_mod <- cmdstan_model("source/bayesian_regression.stan")

blr_fit <- blr_mod$sample(
  chains=2,
  seed=seed,
  refresh=n_samples,
  iter_warmup=1000,
  iter_sampling=n_samples,
  data=train_data
)
```

MCMC MA(3) Model

- MA(3) Model STAN Code for Sampling

```
data {
  int<lower=1> N;
  vector[N] y;
}

parameters {
  real mu;
  vector<lower=-1, upper=1>[3] theta;
  real<lower=0> sigma;
}

transformed parameters {
  vector[N] eps;

  // Initial Values
  eps[1] = y[1] - mu;
  eps[2] = y[2] - mu - theta[1] * eps[1];
  eps[3] = y[3] - mu - theta[1] * eps[2] - theta[2] * eps[1];

  // Estimated Errors
  for (t in 4:N) {
    eps[t] = (y[t] - mu - theta[1]*eps[t-1]
                        - theta[2]*eps[t-2]
                        - theta[3]*eps[t-3]);
  }
}
```

```
model {
  // Posterior Distribution
  mu ~ normal(0, 1);
  theta ~ normal(0, 0.5);
  sigma ~ cauchy(0, 1);

  // Likelihood
  y[4:N] ~ normal(mu + theta[1]*eps[3:(N - 1)]
                     + theta[2]*eps[2:(N - 2)]
                     + theta[3]*eps[1:(N - 3)],
                  sigma);
}

generated quantities {
  // Posterior Predictive Samples
  array[N] real y_post_pred;

  // Initial
  y_post_pred[1] = normal_rng(mu, sigma);
  y_post_pred[2] = normal_rng(mu + theta[1]*eps[1], sigma);
  y_post_pred[3] = normal_rng(mu + theta[1]*eps[2] + theta[2]*eps[3], sigma);
  y_post_pred[4:N] = normal_rng(mu + theta[1]*eps[3:(N - 1)]
                                   + theta[2]*eps[2:(N - 2)]
                                   + theta[3]*eps[1:(N - 3)],
                                   sigma);
}
```

- Associated R Code for MCMC Sampling of MA(3) Model in `MCMC_ma3.Rmd`

```r
source("source/data_sources_and_dependencies.R")

n_samples = 10000
train_data=list(
  N = length(diff_y_scaled),
  y = diff_y_scaled
)

ma3_mod <- cmdstan_model("source/ma3.stan");

ma3_fit <- ma3_mod$sample(
  seed=2026,
  chains=2,
  refresh=n_samples,
  iter_warmup=2500, # Ensures that chain is sufficiently burned-in
  iter_sampling=n_samples,
  data=train_data,
  show_exceptions=FALSE
)
```